

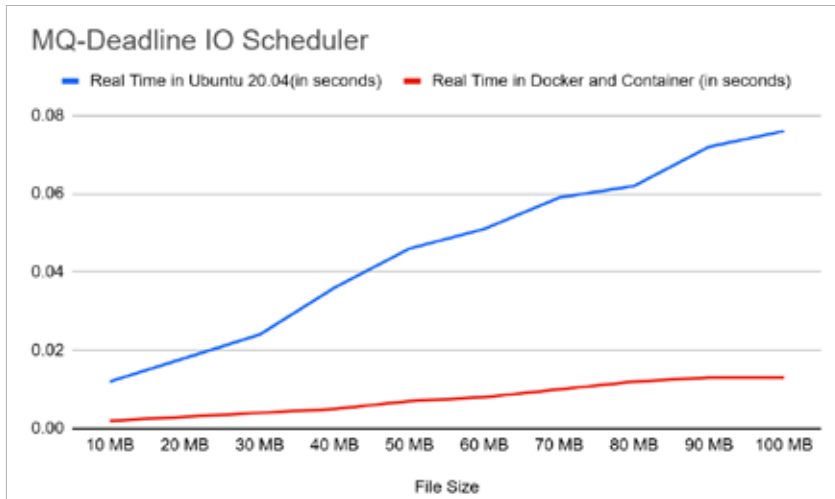


Figure 2: Graphical performance comparison of *mq-deadline* I/O schedulers in VM vs Docker container

File size	Real-time in Ubuntu 20.04 (in seconds)	Real-time in Docker and container (in seconds)	Percentage improvement
10MB	0.012	0.002	83.33
20MB	0.017	0.003	82.35
30MB	0.023	0.004	82.61
40MB	0.038	0.005	86.84
50MB	0.038	0.007	81.58
60MB	0.052	0.008	84.62
70MB	0.058	0.01	82.76
80MB	0.092	0.012	86.96
90MB	0.122	0.013	89.34
100MB	0.128	0.014	89.06

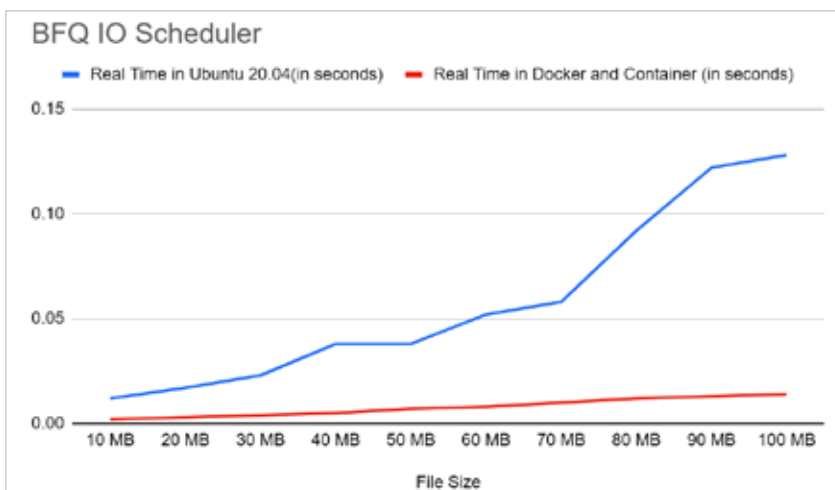


Figure 3: Graphical performance comparison of *bfq* I/O schedulers in VM vs Docker container

take to read a file of various sizes.

```
echo none
while true; do dd if=/dev/zero of=file
bs=1M; done &
echo none > /sys/block/sda/queue/
scheduler
time cat 10MB.txt > /dev/null
time cat 20MB.txt > /dev/null
time cat 30MB.txt > /dev/null
time cat 40MB.txt > /dev/null
time cat 50MB.txt > /dev/null
time cat 60MB.txt > /dev/null
time cat 70MB.txt > /dev/null
time cat 80MB.txt > /dev/null
time cat 90MB.txt > /dev/null
time cat 100MB.txt > /dev/null
```

We executed the read and write operations test cases in a virtual machine and Docker container, and the results are tabulated in Tables 1 to 4 for various schedulers.

Figure 1 shows the performance comparison of *none* I/O schedulers in a virtual machine versus a Docker container.

Figure 2 shows the performance comparison of *mq-deadline* I/O schedulers in a virtual machine versus a Docker container.

Figure 3 shows the performance comparison of *bfq* I/O schedulers in a virtual machine versus a Docker container.

Figure 4 compares the performance of *kyber* I/O schedulers in VM vs Docker container.

Test 2: Effects of high read latency

This benchmark checks how well the I/O scheduler works when reading several small files while also reading a large file.

The code for changing to *none* scheduler and then running the Benchmark Test 2 is given below.

Step 1: Perform the read operation in the background.

Step 2: Record the time taken to read every file in the Android SDK.